

Rapport technique — Avis et alertes Montréal (Projet 3)

Ait Hadi Hadjer

1. Qualités et faiblesses du code

Le projet est divisé en backend (*routes/*, *db/*, *utils/*) et frontend (*components/*, *hooks/*, *pages/*, *services/*), ce qui le rend facile à suivre. Le hook *useAbonnementPush.js* contient toute la logique d'abonnement (permission, abonnement, erreurs), donc *ModaleAbonnement.jsx* reste un composant simple, juste de l'affichage.

Le backend répond toujours de la même façon : `{ data }` pour un succès, `{ error, message }` pour une erreur, avec les bons codes HTTP. *sendNotification.js* supprime aussi automatiquement les abonnements expirés (410/404).

Côté faiblesse, je n'ai aucun test automatisé, tout a été vérifié à la main avec curl. Ça marche pour ce TP mais ça ne protège rien si je retouche le code plus tard. *Accueil.jsx* gère aussi beaucoup de state en même temps (recherche, filtres, dates, pagination, modale) — il pourrait être découpé. Et le backend ne valide pas vraiment ce qu'il reçoit, donc une requête mal formée peut juste planter en erreur 500 au lieu d'un message clair.

2. Qualités et faiblesses de l'application

Les filtres, la recherche et la pagination numérotée fonctionnent bien ensemble. La modale d'abonnement affiche un message clair pour chacun des 3 cas limites (refusée, bloquée, API indisponible).

L'audit Lighthouse (PWA) passe tous les critères : installabilité, écran de démarrage, couleur thématique, icône maskable. Le filtrage par préférences (sujet/arrondissement) fonctionne aussi, testé avec de faux abonnements via curl. Hors-ligne, une bannière avertit l'utilisateur pendant que les dernières alertes restent visibles.

Faiblesse principale : impossible de changer ses préférences sans se désabonner puis se réabonner. Le projet n'est pas déployé non plus, donc tout reste en local pour l'instant.

3. Propositions d'amélioration

- 1 **Priorité haute** — ajouter un bouton « Modifier mes préférences » pour éviter de se désabonner à chaque changement.
- 2 **Priorité moyenne** — ajouter des tests automatisés (Vitest) sur les routes backend critiques, surtout *subscribe* et *send-notification*.
- 3 **Priorité basse** — déployer le frontend sur Vercel et le backend sur Render pour pouvoir faire une démo en ligne.
- 4 **Priorité moyenne** — implémenter une vraie mise en cache (StaleWhileRevalidate) des données de l'API /avis-alertes via le service worker, pour que les alertes restent consultables même lors d'un chargement de page hors-ligne.